

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	REST vs. RESTful: Diseño e Implementación de Servicios Distribuidos				
NÚMERO DE LA PRÁCTICA:	06	AÑO LECTIVO:	2026	NRO. SEMESTRE:	A
FECHA DE PRESENTACIÓN:	25/05/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(S): - Bedregal Perez Daniel - Jara Mamani Mariel Alisson - Mestas Zegarra Christian Raul - Noa Camino Yenaro Joel - Sequeiros Condori Luis Gustavo				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS
ENLACE A GITHUB https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/l6
REPLICACIÓN DE EJERCICIOS RESUELTOS Ejercicio Resuelto: API RESTful de Gestión de Productos Siguiendo las indicaciones del docente, se implementó una API RESTful básica utilizando Python y el framework Flask. El backend gestiona un recurso de "productos" permitiendo operaciones GET, POST y DELETE.
<pre> from flask import Flask, jsonify, request app = Flask(__name__) productos = [{"id": 1, "nombre": "Laptop"}, {"id": 2, "nombre": "Mouse"}] @app.route("/productos", methods=["GET"]) def obtener(): return jsonify(productos) @app.route("/productos", methods=["POST"]) def agregar(): data = request.json productos.append(data) return jsonify({"mensaje": "Producto agregado"}), 201 @app.route("/productos/<int:id>", methods=["DELETE"]) def eliminar(id): global productos productos = [p for p in productos if p["id"] != id] return jsonify({"mensaje": "Producto eliminado"}) if __name__ == "__main__": app.run(debug=True) </pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

Para el consumo de la API, se desarrolló un cliente HTML minimalista que emplea la Fetch API de JavaScript para realizar peticiones asíncronas al servidor y renderizar la lista de productos dinámicamente.

```

async function listar() {
  let r = await fetch("http://127.0.0.1:5000/productos");
  let datos = await r.json();
  let lista = document.getElementById("lista");
  lista.innerHTML = "";
  datos.forEach((p) => {
    lista.innerHTML += `<li>${p.nombre}</li>`;
  });
}

```

A continuación se muestra la validación de los endpoints mediante comandos curl, donde se observa la creación y eliminación exitosa de recursos, así como la persistencia temporal en memoria.

```

> curl -X GET http://127.0.0.1:5000/productos
[
  {
    "id": 1,
    "nombre": "Laptop"
  },
  {
    "id": 2,
    "nombre": "Mouse"
  }
]
> curl -X POST -H 'Content-Type: application/json' -d '{"id":3, "nombre":"Teclado"}' http://127.0.0.1:5000/productos
{"mensaje": "Producto agregado"}
> curl -X DELETE http://127.0.0.1:5000/productos/2
{"mensaje": "Producto eliminado"}
> curl -X GET http://127.0.0.1:5000/productos
[
  {
    "id": 1,
    "nombre": "Laptop"
  },
  {
    "id": 3,
    "nombre": "Teclado"
  }
]

```

Figura 1: Validación de la API de Productos mediante CURL.

SOLUCIÓN DE EJERCICIOS PROPUESTOS

Ejercicio 1: API RESTful Biblioteca (Java + Spring Boot)

Se implementó un sistema de gestión bibliotecaria profesional utilizando Spring Boot. La arquitectura se diseñó en capas, separando el modelo de datos, la capa de acceso a datos (repositorio) y la capa de exposición del servicio (controlador).

En primer lugar, la clase controladora se anotó con `@RestController` y `@RequestMapping` para definir la raíz de los endpoints de libros. Durante la inicialización del controlador, se crea el directorio local de subida de imágenes en caso de que no exista:

```

@RestController
@RequestMapping("/api/books")
@CrossOrigin(origins = "**")
public class BookApiController {

  private final BookRepository bookRepository;
  private final Path rootPath = Paths.get("uploads");

  @Autowired
  public BookApiController(BookRepository bookRepository) {
    this.bookRepository = bookRepository;
    try {
      if (!Files.exists(rootPath)) {
        Files.createDirectories(rootPath);
      }
    } catch (IOException e) {
      throw new RuntimeException("Could not initialize folder for upload!", e);
    }
  }
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

Para la creación y registro de libros (operación POST), se implementó soporte para peticiones de tipo multipart/form-data. Esto permite procesar metadatos en combinación con una imagen binaria (portada). Se valida que los atributos esenciales no estén vacíos, se asegura la unicidad del ISBN mediante consultas al repositorio y se guarda el archivo localmente generando un identificador único para evitar colisiones:

```
@PostMapping(consumes = { "multipart/form-data" })
public ResponseEntity<?> registerBook(
    @RequestParam("title") String title,
    @RequestParam("author") String author,
    @RequestParam("isbn") String isbn,
    @RequestParam(value = "description", required = false) String description,
    @RequestParam(value = "imageUrl", required = false) String imageUrl,
    @RequestParam(value = "image", required = false) MultipartFile image
) {
    if (title == null || title.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("Title is required");
    }
    if (author == null || author.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("Author is required");
    }
    if (isbn == null || isbn.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("ISBN is required");
    }

    if (bookRepository.findByIsbn(isbn.trim()).isPresent()) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body(
            "Book with this ISBN is already registered"
        );
    }

    Book book = new Book();
    book.setTitle(title.trim());
    book.setAuthor(author.trim());
    book.setIsbn(isbn.trim());
    book.setDescription(description != null ? description.trim() : null);

    if (image != null && !image.isEmpty()) {
        try {
            String originalFilename = image.getOriginalFilename();
            String cleanFilename =
                originalFilename != null
                ? originalFilename.replaceAll("[^a-zA-Z0-9.-]", "_")
                : "image";
            String fileName = UUID.randomUUID().toString() + "_" + cleanFilename;

            Files.copy(image.getInputStream(), this.rootPath.resolve(fileName));
            book.setImageUrl("/uploads/" + fileName);
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(
                "Could not upload the image: " + e.getMessage()
            );
        }
    }
    else if (imageUrl != null && !imageUrl.trim().isEmpty()) {
        book.setImageUrl(imageUrl.trim());
    }

    Book savedBook = bookRepository.save(book);
    return ResponseEntity.status(HttpStatus.CREATED).body(savedBook);
}
```

El modelo de datos correspondiente a los libros se definió mediante una clase JPA (Book), mapeándola a una tabla relacional en una base de datos SQLite. Los atributos clave se decoraron con restricciones de integridad como nullable = false y unique = true para el ISBN:

```
package com.lab06.e1.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```
import jakarta.persistence.Id;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String author;

    @Column(nullable = false, unique = true)
    private String isbn;

    @Column(length = 2000)
    private String description;

    private String imageUrl;
}
```

La validación de la API se realizó mediante pruebas de carga de datos, verificando que las restricciones de unicidad del ISBN y el manejo de archivos multipart funcionen correctamente según los principios RESTful.

```
> curl -s http://localhost:8080/api/books | jq '.' | head -n 30
[
  {
    "id": 1,
    "title": "1984",
    "author": "George Orwell",
    "isbn": "0451524934",
    "description": "A dystopian novel set in a totalitarian society ruled by Big Brother, exploring themes of surveillance, control, and truth.",
    "imageUrl": "https://covers.openlibrary.org/b/isbn/0451524934-L.jpg"
  },
  {
    "id": 2,
    "title": "To Kill a Mockingbird",
    "author": "Harper Lee",
    "isbn": "0446310786",
    "description": "The story of young Scout Finch and her father Atticus, a lawyer defending a Black man falsely accused of rape in the segregated South.",
    "imageUrl": "https://covers.openlibrary.org/b/isbn/0446310786-L.jpg"
  },
  {
    "id": 3,
    "title": "The Great Gatsby",
    "author": "F. Scott Fitzgerald",
    "isbn": "0743273567",
    "description": "A portrait of the Jazz Age, following the mysterious millionaire Jay Gatsby and his tragic obsession with the beautiful Daisy Buchanan.",
    "imageUrl": "https://covers.openlibrary.org/b/isbn/0743273567-L.jpg"
  },
  {
    "id": 4,
    "title": "Project Hail Mary",
    "author": "Andy Weir",
    "isbn": "0593230574",
  }
]
```

Figura 2: Pruebas de la API de Biblioteca mediante CURL.

Se desarrolló una interfaz web moderna y responsiva para interactuar con la biblioteca de forma intuitiva, permitiendo el registro y edición de libros mediante formularios dinámicos.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

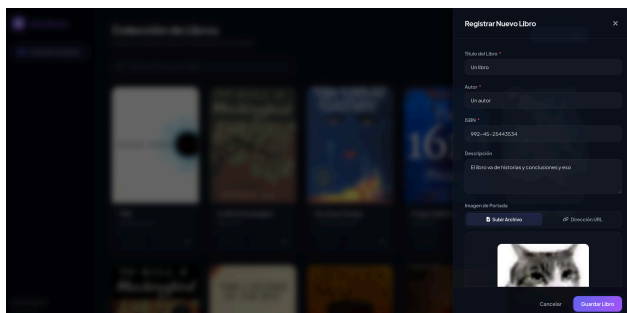


Figura 3: Formulario de creación para libros.

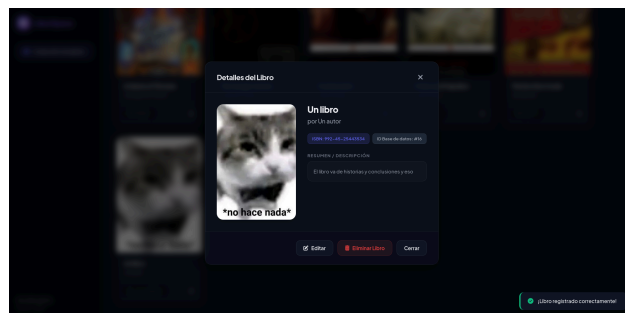


Figura 4: Resultado de crear un libro y mostrar sus datos.

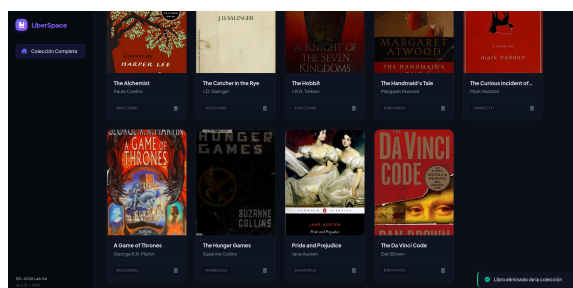


Figura 5: Resultado de eliminar un libro en el listado.

Ejercicio 2: API RESTful Estudiantes (Python)

Este ejercicio consistió en la creación de un sistema de registro estudiantil avanzado empleando Flask y SQLAlchemy. La persistencia de datos se gestiona a través de un modelo ORM que mapea la entidad Estudiante a una tabla en SQLite, incluyendo campos para métricas académicas y estados de matriculación:

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class Estudiante(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    matricula = db.Column(db.String(50), nullable=False)
    nombre = db.Column(db.String(100), nullable=False)
    carrera = db.Column(db.String(100), nullable=False)
    edad = db.Column(db.Integer, nullable=False)
    email = db.Column(db.String(120), nullable=False)
    telefono = db.Column(db.String(30), nullable=False)
    matriculado = db.Column(db.Boolean, nullable=False, default=True)
    semestres = db.Column(db.Integer, nullable=False)
    estado = db.Column(db.String(40), nullable=False)

    def to_dict(self):
        return {
            "id": self.id,
            "matricula": self.matricula,
            "nombre": self.nombre,
            "carrera": self.carrera,
            "edad": self.edad,
            "email": self.email,
            "telefono": self.telefono,
            "matriculado": self.matriculado,
            "semestres": self.semestres,
            "estado": self.estado,
        }
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

Las rutas y controladores de la API se implementaron utilizando decoradores de Flask sobre funciones orientadas a recursos. A continuación, se presenta el endpoint GET que consulta todos los estudiantes de la base de datos a través de la capa ORM y los serializa a formato JSON para su transmisión:

```
@app.route("/estudiantes", methods=["GET"])
def listar():
    estudiantes = Estudiante.query.all()
    return jsonify([e.to_dict() for e in estudiantes])
```

Para la inserción de nuevos registros académicos (operación POST), se extrae el cuerpo en formato JSON de la petición HTTP, se realiza la coerción y validación de tipos necesarios (como el estado de matriculado y semestres) y se guarda la nueva entidad en la base de datos de manera atómica:

```
@app.route("/estudiantes", methods=["POST"])
def agregar():
    data = request.json
    nuevo = Estudiante(
        matricula=data.get("matricula", ""),
        nombre=data.get("nombre", ""),
        carrera=data.get("carrera", ""),
        edad=data.get("edad", 0),
        email=data.get("email", ""),
        telefono=data.get("telefono", ""),
        matriculado=bool(data.get("matriculado", True))
        if isinstance(data.get("matriculado", True), bool)
        else data.get("matriculado", "true") in ["true", True, 1],
        semestres=data.get("semestres", 0),
        estado=data.get("estado", "Activo"),
    )
    db.session.add(nuevo)
    db.session.commit()
    return jsonify({"ok": True})
```

Por último, la eliminación de un recurso individual se realiza mediante el verbo DELETE, especificando el ID único del estudiante directamente en la URI. El servidor busca el estudiante y remueve su registro de SQLite, retornando una respuesta semántica:

```
@app.route("/estudiantes/<int:i>", methods=["DELETE"])
def eliminar(i):
    estudiante = db.session.get(Estudiante, i)
    if estudiante:
        db.session.delete(estudiante)
        db.session.commit()
        return jsonify({"eliminado": True})
    return jsonify({"error": "Estudiante no encontrado"}), 404
```

A continuación se presenta la ejecución del cliente consumidor, el cual automatiza una serie de peticiones HTTP para validar el ciclo de vida completo de un recurso estudiantil en el servidor.

```
> curl -s http://localhost:5000/estudiantes | jq '.' | head -n 30
[
  {
    "carrera": "Ingeniería de Sistemas",
    "edad": 21,
    "email": "carlos.mendoza@campus.edu",
    "estado": "Activo",
    "id": 1,
    "matricula": "2024-IS-0142",
    "matriculado": true,
    "nombre": "Carlos Mendoza",
    "semestres": 5,
    "telefono": "+51 999 120 410"
  },
  {
    "carrera": "Medicina",
    "edad": 23,
    "email": "lucia.fernandez@campus.edu",
    "estado": "Activo",
    "id": 2,
    "matricula": "2023-MED-0098",
    "matriculado": true,
    "nombre": "Lucia Fernandez",
    "semestres": 8,
    "telefono": "+51 988 204 118"
  },
  {
    "carrera": "Arquitectura",
    "edad": 19,
    "email": "javier.solis@campus.edu",
    "estado": "En riesgo",
    "id": 3,
    "matricula": "2022-ARQ-0056",
    "matriculado": false,
    "nombre": "Javier Solis",
    "semestres": 2,
    "telefono": "+51 977 301 902"
  },
  {
    "carrera": "Diseño Gráfico",
    "edad": 22,
    "email": "mariana.ros@campus.edu",
    "estado": "Activo",
    "id": 4,
    "matricula": "2021-DGR-0015",
    "matriculado": true,
    "nombre": "Mariana Ros",
    "semestres": 7,
    "telefono": "+51 982 227 585"
  }
]
```

Figura 6: Ejecución del cliente de pruebas para la API de Estudiantes.

Finalmente, se diseñó un panel administrativo (Dashboard) que ofrece visualizaciones estadísticas sobre el alumnado y herramientas de filtrado avanzado para la gestión de registros.

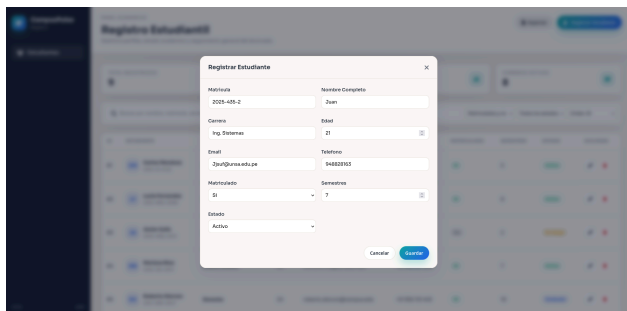
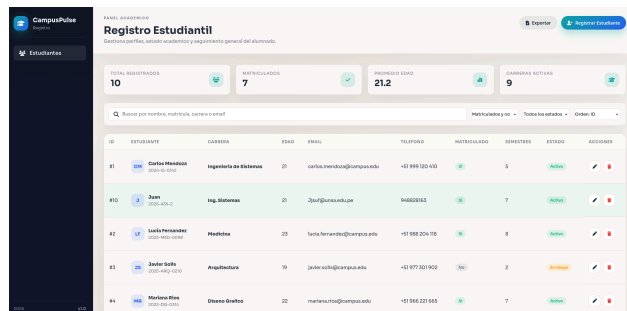
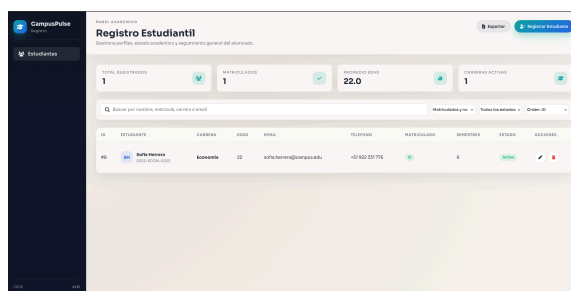


Figura 7: Formulario de creación para estudiantes.



ID	ESTUDIANTE	CARRERA	EDAD	EMAIL	TELEFONO	MATRICULADO	SEMESTRES	ESTADO	ACCIONES
01	Carlos Mendoza	Ingeniería de Sistemas	21	carlos.mendoza@campus.edu	+51 999 120 410	SI	5	Activo	[icon]
02	Lucia Fernandez	Medicina	23	lucia.fernandez@campus.edu	+51 988 204 118	SI	8	Activo	[icon]
03	Javier Solis	Arquitectura	19	javier.solis@campus.edu	+51 977 301 902	NO	2	En Riesgo	[icon]
04	Mariana Ros	Diseño Gráfico	22	mariana.ros@campus.edu	+51 982 227 585	SI	7	Activo	[icon]

Figura 8: Resultado de crear un estudiante y ver sus datos.



ID	ESTUDIANTE	CARRERA	EDAD	EMAIL	TELEFONO	MATRICULADO	SEMESTRES	ESTADO	ACCIONES
05	Sofia Herrera	Economía	22	sofia.herrera@campus.edu	+51 982 203 776	SI	6	Activo	[icon]

Figura 9: Resultado de eliminar todos los estudiantes del listado.

CUESTIONARIO

¿Por qué una API que utiliza HTTP no necesariamente puede considerarse RESTful?

Porque RESTful no se define simplemente por el uso del protocolo HTTP, sino por el cumplimiento estricto de las restricciones arquitectónicas de REST (Representational State Transfer). Muchas APIs utilizan HTTP solo como un túnel para RPC (Remote Procedure Call), donde los endpoints se nombran según acciones (/crearUsuario, /obtenerDatos) en lugar de recursos, y no respetan la semántica de los verbos HTTP ni la naturaleza sin estado o la interfaz uniforme (HATEOAS).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

¿Qué consecuencias tendría diseñar endpoints orientados a acciones y no a recursos en sistemas distribuidos escalables?

Diseñar endpoints orientados a acciones (GET /deleteUser?id=5) rompe la uniformidad de la interfaz y dificulta la cacheabilidad. En sistemas distribuidos, esto genera una explosión de URLs difíciles de documentar y mantener. Además, impide que intermediarios (como proxies o balanceadores) optimicen las peticiones basándose en la semántica estándar de HTTP, limitando la escalabilidad horizontal y aumentando el acoplamiento entre cliente y servidor.

Compare RESTful frente a RPC y explique en qué escenarios empresariales RESTful podría ser una mala elección.

RESTful se centra en recursos y estado (escalable, desacoplado), mientras que RPC (como gRPC o RMI) se centra en procedimientos (eficiente, contrato fuerte). RESTful podría ser una mala elección en escenarios de baja latencia extrema (comunicación entre microservicios de alto rendimiento), donde el overhead de JSON y las cabeceras HTTP es excesivo. También es inadecuado para sistemas con flujos de datos complejos y multidireccionales que se benefician más de un contrato binario estricto y streaming.

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

1. La arquitectura RESTful proporciona un estándar claro para la interoperabilidad en sistemas distribuidos, permitiendo que clientes heterogéneos consuman servicios mediante una interfaz uniforme.
2. La distinción entre REST (modelo teórico) y RESTful (implementación práctica) es fundamental para desarrollar APIs que realmente aprovechen las capacidades de optimización del protocolo HTTP.
3. El uso de frameworks modernos como Spring Boot y Flask simplifica enormemente la aplicación de principios RESTful, facilitando la gestión de estados y la serialización de datos.

RECOMENDACIONES

1. Se recomienda priorizar siempre el diseño orientado a recursos sobre el orientado a verbos para asegurar que la API sea intuitiva, documentable y compatible con estándares de industria.
2. Es fundamental utilizar correctamente los códigos de estado HTTP (200, 201, 404, 500, etc.) para que el cliente pueda manejar los errores y respuestas de forma semántica y predecible.
3. Para APIs destinadas a producción, se aconseja implementar mecanismos de autenticación (JWT) y limitación de tasa (rate limiting) para proteger los recursos expuestos.

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
- [2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
- [3] Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.
- [4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: Alfaomega Ra-Ma.
- [5] Fielding, R. (2000). Architectural Styles and the Design of Network-based Software Architectures. Dissertation. University of California, Irvine.
- [6] Pautasso, O., Zimmermann, O., & Leymann, F. (2008). RESTful Web Services vs. "Big" Web Services: Making the Right Architectural Decision. WWW '08.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

ANEXOS

Ejercicio 1: API RESTful Biblioteca (Java + Spring Boot)

Aplicación Principal (Spring Boot)

```
package com.lab06.e1;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ElApplication {

    public static void main(String[] args) {
        SpringApplication.run(ElApplication.class, args);
    }

}
```

Modelo del Libro (Entidad JPA)

```
package com.lab06.e1.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Column;
import lombok.Getter;
import lombok.Setter;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;

@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String author;

    @Column(nullable = false, unique = true)
    private String isbn;

    @Column(length = 2000)
    private String description;

    private String imageUrl;
}
```

Repositorio de Libros (Spring Data JPA)

```
package com.lab06.e1.repository;

import com.lab06.e1.model.Book;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```
import java.util.Optional;

@Repository
public interface BookRepository extends JpaRepository<Book, Long> {
    Optional<Book> findByIsbn(String isbn);
}
```

Controlador de la API REST

```
package com.lab06.e1.controller;

import com.lab06.e1.model.Book;
import com.lab06.e1.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.List;
import java.util.Optional;
import java.util.UUID;

@RestController
@RequestMapping("/api/books")
@CrossOrigin(origins = "**")
public class BookApiController {

    private final BookRepository bookRepository;
    private final Path rootPath = Paths.get("uploads");

    @Autowired
    public BookApiController(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
        try {
            if (!Files.exists(rootPath)) {
                Files.createDirectories(rootPath);
            }
        } catch (IOException e) {
            throw new RuntimeException("Could not initialize folder for upload!", e);
        }
    }

    @GetMapping
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getBookById(@PathVariable Long id) {
        return bookRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping(consumes = {"multipart/form-data"})
    public ResponseEntity<?> registerBook(
        @RequestParam("title") String title,
        @RequestParam("author") String author,
        @RequestParam("isbn") String isbn,
        @RequestParam(value = "description", required = false) String description,
        @RequestParam(value = "imageUrl", required = false) String imageUrl,
        @RequestParam(value = "image", required = false) MultipartFile image) {

        if (title == null || title.trim().isEmpty()) {
            return ResponseEntity.badRequest().body("Title is required");
        }
    }
}
```

```
if (author == null || author.trim().isEmpty()) {
    return ResponseEntity.badRequest().body("Author is required");
}
if (isbn == null || isbn.trim().isEmpty()) {
    return ResponseEntity.badRequest().body("ISBN is required");
}

// Check if ISBN already exists
if (bookRepository.findByIsbn(isbn.trim()).isPresent()) {
    return ResponseEntity.status(HttpStatus.CONFLICT).body("Book with this ISBN is already registered");
}

Book book = new Book();
book.setTitle(title.trim());
book.setAuthor(author.trim());
book.setIsbn(isbn.trim());
book.setDescription(description != null ? description.trim() : null);

if (image != null && !image.isEmpty()) {
    try {
        String originalFilename = image.getOriginalFilename();
        String cleanFilename = originalFilename != null ? originalFilename.replaceAll("[^a-zA-Z0-9.-]", "_") : "image";
        String fileName = UUID.randomUUID().toString() + "_" + cleanFilename;

        Files.copy(image.getInputStream(), this.rootPath.resolve(fileName));
        book.setImageUrl("/uploads/" + fileName);
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Could not upload the image: " + e.getMessage());
    }
} else if (imageUrl != null && !imageUrl.trim().isEmpty()) {
    book.setImageUrl(imageUrl.trim());
}

Book savedBook = bookRepository.save(book);
return ResponseEntity.status(HttpStatus.CREATED).body(savedBook);
}

@PutMapping(value =("/{id}", consumes = {"multipart/form-data"})
public ResponseEntity<?> updateBook(
    @PathVariable Long id,
    @RequestParam("title") String title,
    @RequestParam("author") String author,
    @RequestParam("isbn") String isbn,
    @RequestParam(value = "description", required = false) String description,
    @RequestParam(value = "imageUrl", required = false) String imageUrl,
    @RequestParam(value = "image", required = false) MultipartFile image) {

    Optional<Book> bookOptional = bookRepository.findById(id);
    if (bookOptional.isEmpty()) {
        return ResponseEntity.notFound().build();
    }
    Book book = bookOptional.get();

    if (title == null || title.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("Title is required");
    }
    if (author == null || author.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("Author is required");
    }
    if (isbn == null || isbn.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("ISBN is required");
    }

    // Check if ISBN is taken by another book
    Optional<Book> existingIsbn = bookRepository.findByIsbn(isbn.trim());
    if (existingIsbn.isPresent() && !existingIsbn.get().getId().equals(id)) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body("Book with this ISBN is already registered");
    }

    book.setTitle(title.trim());
    book.setAuthor(author.trim());
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

```

book.setIsbn(isbn.trim());
book.setDescription(description != null ? description.trim() : null);

if (image != null && !image.isEmpty()) {
    // Case A: New file uploaded. Remove old disk image if there was one.
    deleteLocalImage(book.getImageUrl());
    try {
        String originalFilename = image.getOriginalFilename();
        String cleanFilename = originalFilename != null ? originalFilename.replaceAll("[^a-zA-Z0-9.-]", "_") : "image";
        String fileName = UUID.randomUUID().toString() + "_" + cleanFilename;

        Files.copy(image.getInputStream(), this.rootPath.resolve(fileName));
        book.setImageUrl("/uploads/" + fileName);
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Could not upload the image: " + e.getMessage());
    }
} else if (imageUrl != null) {
    String trimmedUrl = imageUrl.trim();
    if (trimmedUrl.isEmpty()) {
        // User explicitly cleared the cover image
        deleteLocalImage(book.getImageUrl());
        book.setImageUrl(null);
    } else if (!trimmedUrl.equals(book.getImageUrl())) {
        // Case B: A new public URL is specified, remove old local image
        deleteLocalImage(book.getImageUrl());
        book.setImageUrl(trimmedUrl);
    }
}

Book updatedBook = bookRepository.save(book);
return ResponseEntity.ok(updatedBook);
}

@DeleteMapping("/{id}")
public ResponseEntity<?> deleteBook(@PathVariable Long id) {
    Optional<Book> bookOptional = bookRepository.findById(id);
    if (bookOptional.isEmpty()) {
        return ResponseEntity.notFound().build();
    }

    Book book = bookOptional.get();
    deleteLocalImage(book.getImageUrl());
    bookRepository.delete(book);
    return ResponseEntity.ok().body("Book deleted successfully");
}

private void deleteLocalImage(String imageUrl) {
    if (imageUrl != null && imageUrl.startsWith("/uploads/")) {
        String filename = imageUrl.substring("/uploads/".length());
        try {
            Path filePath = this.rootPath.resolve(filename);
            Files.deleteIfExists(filePath);
        } catch (IOException e) {
            System.err.println("Could not delete file " + filename + ": " + e.getMessage());
        }
    }
}
}

```

Configuración CORS (WebConfig)

```

package com.lab06.e1.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import java.nio.file.Path;
import java.nio.file.Paths;

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        Path uploadDir = Paths.get("uploads");
        String uploadPath = uploadDir.toFile().getAbsolutePath();

        // Ensure standard file protocol prefix is present
        String resourceLocation = "file:" + uploadPath + "/";

        registry.addResourceHandler("/uploads/**")
            .addResourceLocations(resourceLocation);
    }
}
```

Poblado Inicial de Base de Datos (DatabaseSeeder)

```
package com.lab06.e1.config;

import com.lab06.e1.model.Book;
import com.lab06.e1.repository.BookRepository;
import java.io.File;
import java.util.List;
import java.util.Map;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
import tools.jackson.core.type.TypeReference;
import tools.jackson.databind.ObjectMapper;

@Component
public class DatabaseSeeder implements CommandLineRunner {

    private final BookRepository bookRepository;
    private final ObjectMapper objectMapper;

    @Autowired
    public DatabaseSeeder(BookRepository bookRepository, ObjectMapper objectMapper) {
        this.bookRepository = bookRepository;
        this.objectMapper = objectMapper;
    }

    @Override
    public void run(String... args) throws Exception {
        File seedFile = new File("seed/data.json");

        if (!seedFile.exists()) {
            System.out.println(
                "Seeder: seed/data.json file not found at " +
                seedFile.getAbsolutePath() +
                ". Skipping database seeding."
            );
            return;
        }

        System.out.println(
            "Seeder: Found seed file at " + seedFile.getAbsolutePath() + ". Starting seeding process..."
        );

        try {
            List<Map<String, Object>> seedBooks = objectMapper.readValue(
                seedFile,
                new TypeReference<List<Map<String, Object>>>() {}
            );

            int seededCount = 0;
            for (Map<String, Object> map : seedBooks) {
                String isbn = (String) map.get("isbn");
                if (isbn != null && !isbn.trim().isEmpty()) {
                    isbn = isbn.trim();
                }
            }
        }
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

```

    if (bookRepository.findByIsbn(isbn).isEmpty()) {
        Book book = new Book();
        book.setIsbn(isbn);
        book.setTitle(((String) map.get("title")).trim());
        book.setAuthor(((String) map.get("author")).trim());

        String description = (String) map.get("description");
        book.setDescription(description != null ? description.trim() : null);

        String imageUrl = (String) map.get("image_url");
        book.setImageUrl(imageUrl != null ? imageUrl.trim() : null);

        bookRepository.save(book);
        seededCount++;
    }
}

System.out.println(
    "Seeder: Seeding process finished. Added " + seededCount + " new books to the database."
);
} catch (Exception e) {
    System.err.println("Seeder: Error occurred during seeding process: " + e.getMessage());
    e.printStackTrace();
}
}
}

```

Ejercicio 2: API RESTful Estudiantes (Python + Flask)

Archivo Principal de Inicio (main.py)

```

from app import create_app

app = create_app()

if __name__ == '__main__':
    app.run(debug=True, port=5000)

```

Inicialización de la Aplicación y SQLAlchemy (__init__.py)

```

import os
import json
from flask import Flask
from .models import db, Estudiante

def create_app():
    app = Flask(__name__, static_url_path='')

    # Configuración de SQLite
    basedir = os.path.abspath(os.path.dirname(__file__))

    # Ubicación de la base de datos
    app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:/// ' + os.path.join(basedir, '..', 'estudiantes.db')
    app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

    db.init_app(app)

    from .routes import register_routes
    register_routes(app)

    # Llenar la base de datos con datos iniciales si está vacía
    with app.app_context():
        db.create_all()
        if Estudiante.query.first() is None:
            seed_file_path = os.path.join(basedir, '..', 'seed', 'data-mock.json')
            if os.path.exists(seed_file_path):
                with open(seed_file_path, 'r', encoding='utf-8') as f:
                    data = json.load(f)

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

```

datos_iniciales = data.get('datos_iniciales', [])
for est_data in datos_iniciales:
    nuevo = Estudiante(
        matricula=est_data['matricula'],
        nombre=est_data['nombre'],
        carrera=est_data['carrera'],
        edad=est_data['edad'],
        email=est_data['email'],
        telefono=est_data['telefono'],
        matriculado=est_data['matriculado'],
        semestres=est_data['semestres'],
        estado=est_data['estado']
    )
    db.session.add(nuevo)
db.session.commit()
print("✅ Base de datos inicializada desde seed/data-mock.json")

return app

```

Modelo del Estudiante (SQLAlchemy)

```

from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class Estudiante(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    matricula = db.Column(db.String(50), nullable=False)
    nombre = db.Column(db.String(100), nullable=False)
    carrera = db.Column(db.String(100), nullable=False)
    edad = db.Column(db.Integer, nullable=False)
    email = db.Column(db.String(120), nullable=False)
    telefono = db.Column(db.String(30), nullable=False)
    matriculado = db.Column(db.Boolean, nullable=False, default=True)
    semestres = db.Column(db.Integer, nullable=False)
    estado = db.Column(db.String(40), nullable=False)

    def to_dict(self):
        return {
            "id": self.id,
            "matricula": self.matricula,
            "nombre": self.nombre,
            "carrera": self.carrera,
            "edad": self.edad,
            "email": self.email,
            "telefono": self.telefono,
            "matriculado": self.matriculado,
            "semestres": self.semestres,
            "estado": self.estado
        }

```

Definición de Rutas de la API (routes.py)

```

from flask import request, jsonify, send_from_directory
from .models import db, Estudiante

def register_routes(app):

    @app.route('/')
    def index():
        return send_from_directory(app.static_folder, 'index.html')

    @app.route('/estudiantes', methods=['GET'])
    def listar():
        estudiantes = Estudiante.query.all()
        return jsonify([e.to_dict() for e in estudiantes])

    @app.route('/estudiantes', methods=['POST'])
    def agregar():

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

```

data = request.json
nuevo = Estudiante(
    matricula=data.get('matricula', ''),
    nombre=data.get('nombre', ''),
    carrera=data.get('carrera', ''),
    edad=data.get('edad', 0),
    email=data.get('email', ''),
    telefono=data.get('telefono', ''),
    matriculado=bool(data.get('matriculado', True)) if isinstance(data.get('matriculado', True), bool) else
data.get('matriculado', 'true') in ['true', True, 1],
    semestres=data.get('semestres', 0),
    estado=data.get('estado', 'Activo')
)
db.session.add(nuevo)
db.session.commit()
return jsonify({"ok": True})

@app.route('/estudiantes/<int:i>', methods=['PUT'])
def actualizar(i):
    estudiante = db.session.get(Estudiante, i)
    if estudiante:
        data = request.json
        if 'matricula' in data:
            estudiante.matricula = data['matricula']
        if 'nombre' in data:
            estudiante.nombre = data['nombre']
        if 'carrera' in data:
            estudiante.carrera = data['carrera']
        if 'edad' in data:
            estudiante.edad = data['edad']
        if 'email' in data:
            estudiante.email = data['email']
        if 'telefono' in data:
            estudiante.telefono = data['telefono']
        if 'matriculado' in data:
            estudiante.matriculado = data['matriculado'] if isinstance(data['matriculado'], bool) else data['matriculado'] in
['true', True, 1]
        if 'semestres' in data:
            estudiante.semestres = data['semestres']
        if 'estado' in data:
            estudiante.estado = data['estado']

        db.session.commit()
        return jsonify({"actualizado": True})
    return jsonify({"error": "Estudiante no encontrado"}), 404

@app.route('/estudiantes/<int:i>', methods=['DELETE'])
def eliminar(i):
    estudiante = db.session.get(Estudiante, i)
    if estudiante:
        db.session.delete(estudiante)
        db.session.commit()
        return jsonify({"eliminado": True})
    return jsonify({"error": "Estudiante no encontrado"}), 404

```

Script Cliente de Pruebas (cliente.py)

```

import requests
import json
import os

BASE_URL = 'http://127.0.0.1:5000/estudiantes'

def test_api():
    print("="*60)
    print("🚀 EVIDENCIA DE EJECUCIÓN API RESTFUL Y CLIENTE CONSUMIDOR")
    print("="*60)

    # 1. Registrar estudiante (POST)
    print("\n📌 [POST] Registrando un nuevo estudiante...")

```



```
nuevo_estudiante = {
    "matricula": "2026-TEA-0101",
    "nombre": "Pedro Pascal",
    "carrera": "Artes Escenicas",
    "edad": 28,
    "email": "pedro.pascal@campus.edu",
    "telefono": "+51 955 110 220",
    "matriculado": True,
    "semestres": 4,
    "estado": "Activo"
}
print(f" Enviando datos: {json.dumps(nuevo_estudiante)}")
r_post = requests.post(BASE_URL, json=nuevo_estudiante)
print(f" => Respuesta (Status {r_post.status_code}): {r_post.json()}")

# 2. Consultar estudiantes (GET)
print("\n2 [GET] Consultando lista de estudiantes...")
r_get = requests.get(BASE_URL)
estudiantes = r_get.json()
print(f" => Se obtuvieron {len(estudiantes)} registros.")
print(f" => Muestra de los últimos 2 registros:")
for est in estudiantes[-2:]:
    print(
        f" - ID: {est['id']} | {est['nombre']} | {est['matricula']} | "
        f"{est['carrera']} | {est['edad']} años | {est['estado']}"
    )

if estudiantes:
    # Usamos el ID del último estudiante (que debería ser el que acabamos de crear)
    ultimo_id = estudiantes[-1]['id']

# 3. Actualizar estudiante (PUT)
print(f"\n3 [PUT] Actualizando estudiante con ID {ultimo_id}...")
datos_actualizacion = {
    "nombre": "Pedro Pascal Modificado",
    "edad": 29,
    "matriculado": False,
    "semestres": 5,
    "estado": "En riesgo"
}
print(f" Enviando datos: {json.dumps(datos_actualizacion)}")
r_put = requests.put(f"{BASE_URL}/{ultimo_id}", json=datos_actualizacion)
print(f" => Respuesta (Status {r_put.status_code}): {r_put.json()}")

# Consultar para confirmar actualización
print(f" => Verificando estado tras la actualización...")
r_verify = requests.get(BASE_URL)
estudiante_actualizado = next((e for e in r_verify.json() if e['id'] == ultimo_id), None)
print(f" Resultado en BD: {estudiante_actualizado}")

# 4. Eliminar estudiante (DELETE)
print(f"\n4 [DELETE] Eliminando estudiante con ID {ultimo_id}...")
r_delete = requests.delete(f"{BASE_URL}/{ultimo_id}")
print(f" => Respuesta (Status {r_delete.status_code}): {r_delete.json()}")

# Consultar para confirmar eliminación
print(f" => Verificando estado tras la eliminación...")
r_verify_del = requests.get(BASE_URL)
estudiante_eliminado = next((e for e in r_verify_del.json() if e['id'] == ultimo_id), None)
if estudiante_eliminado is None:
    print(" ✅ Confirmado: El estudiante ya no existe en la BD.")
else:
    print(" ❌ Error: El estudiante aún existe.")
else:
    print("\n⚠️ No hay estudiantes en la BD para probar UPDATE/DELETE.")

print("\n" + "="*60)
print("✅ PRUEBAS FINALIZADAS CON ÉXITO")
print("="*60)

if __name__ == '__main__':
    try:
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

```
# Hacer una prueba de conexión rápida
requests.get(BASE_URL, timeout=2)
test_api()
except requests.exceptions.ConnectionError:
    print(f"❌ ERROR: No se pudo conectar al servidor en {BASE_URL}.")
    print(" Por favor, asegúrate de que el servidor Flask (run.py) esté corriendo.")
    print(" Comando: uv run python run.py")
```